

EventsApp SQL Injection Demonstration & Mitigation (Milestone 4)

Owen Lindsey

Professor Sluiter, Shad

CST-407

Grand Canyon University

Overview

Expose the SQL injection vulnerability in the Events search flow, then refactor the data layer to use prepared statements so user input can no longer alter SQL. The work targets the Spring Boot project at `coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp` (MariaDB/MySQL backend).

Video Demonstration

Link (≤5 minutes): https://youtu.be/d_W3xBXQfAk

Walkthrough: (1) exploit the search form with `%\ ' OR '1'='1' --` to return all rows, (2) inspect the vulnerable concatenated SQL, (3) apply prepared statements in the repositories, and (4) rerun the same payload to show it fails to inject.

1. Identify SQL Injection Points

EventRepository built every query with string concatenation. The search path `(findByDescription)` generated `SELECT * FROM events WHERE description LIKE '%" + description + "%'`, so any injected text could rewrite the WHERE clause. Organizer lookups, `findById`, `deleteById`, and `insert/update` all shared the same pattern. UserRepository repeated the issue for login lookups, ID lookups, and role writes, which could enable authentication or role-manipulation injection. The common root cause: user-controlled strings were placed directly into SQL without binding.

2. Demonstrate the Vulnerability

With the app running (`mvn spring-boot:run`) against `jdbc:mysql://localhost:3306/eventsapp`, open `/events/search` and submit `%\ ' OR '1'='1' --`. The page returns all events because the injected `OR '1'='1'` makes the predicate always true and the trailing comment drops the rest of the SQL.

3. Implement Mitigation

Prepared Statements Everywhere

EventRepository now binds parameters for every method. The search query uses `LIKE ?` with trimmed input:

```
String sql = "SELECT * FROM events WHERE description LIKE ?";
return jdbcTemplate.query(sql, mapper, "%" + searchTerm.trim() + "%");
```

Organizer lookups, ID lookups, existence checks, deletes, and insert/update all use placeholders; insert/update also uses `PreparedStatement` plus `KeyHolder` to capture generated IDs.

`UserRepository` received the same treatment. Login lookups, ID lookups, deletes, inserts/updates, and role CRUD bind parameters instead of concatenating strings; inserts/updates also use `PreparedStatement` + `KeyHolder`.

Input Handling

Search terms are trimmed before binding to `LIKE` to avoid accidental trailing-space wildcards that could broaden results.

4. Test the Application (After Fix)

Manual Verification

Scenario	Payload	Expected (fixed)	Observed
Search injection	<code>%\ ' OR '1'='1' --</code>	Treated as literal text; only real matches (or none) returned	✓ No forced “all rows”
Search noise	<code>%' UNION SELECT 1,2,3,4,5,6 --</code>	Fails or returns none; no data leakage	✓ No UNION injection
Login lookup	<code>admin' --</code> (username)	Treated as literal username; password still required	✓ No bypass

5. Source Code Changes

All SQL statements are now parameterized in both repositories. `EventRepository` binds parameters for search, organizer filters, ID lookups, deletes, and insert/update with `KeyHolder`. `UserRepository` binds parameters for login lookups, ID lookups, deletes, insert/update with `KeyHolder`, and role CRUD. Files:

```
src/main/java/com/shadsluiter/eventsapp/data/EventRepository.java
src/main/java/com/shadsluiter/eventsapp/data/UserRepository.java
```

Repository: https://github.com/omniV1/GCU_SWE_2023-2025/tree/main/CST-407-Application-Security-Foundations/coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp

6. How to Run

```
cd coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp
mvn spring-boot:run
```

Configure DB credentials in `src/main/resources/application.properties` (defaults: `jdbc:mysql://localhost:3306/eventsapp,user root,password root`). MariaDB works with the existing MySQL driver; `jdbc:mariadb://...` is also acceptable.